

Practical - 3

Aim: Utilize core (http, fs, url, util, path, os, events) and local modules to demonstrate their output

1: Create an HTTP server using core modules that responds with different messages based on URL paths:

- a) Respond with “Home Page” for /
- b) Respond with a formatted HTML table for /exam
- c) Respond with a 404 message for any other path.

Code:

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Home Page');
  } else if (req.url === '/exam') {
    res.writeHead(200, { 'Content-Type': 'text/html' });

    res.end(`
      <!DOCTYPE html>
      <html>
      <head>
        <title>Exam Details</title>
        <style>
          table {
            border-collapse: collapse;
            width: 60%;
          }
          th, td {
            border: 1px solid black;
            padding: 10px;
            text-align: center;
          }
          th {
            background-color: #f2f2f2;
          }
        </style>
      </head>
      <body>
        <h1>Exam Schedule</h1>

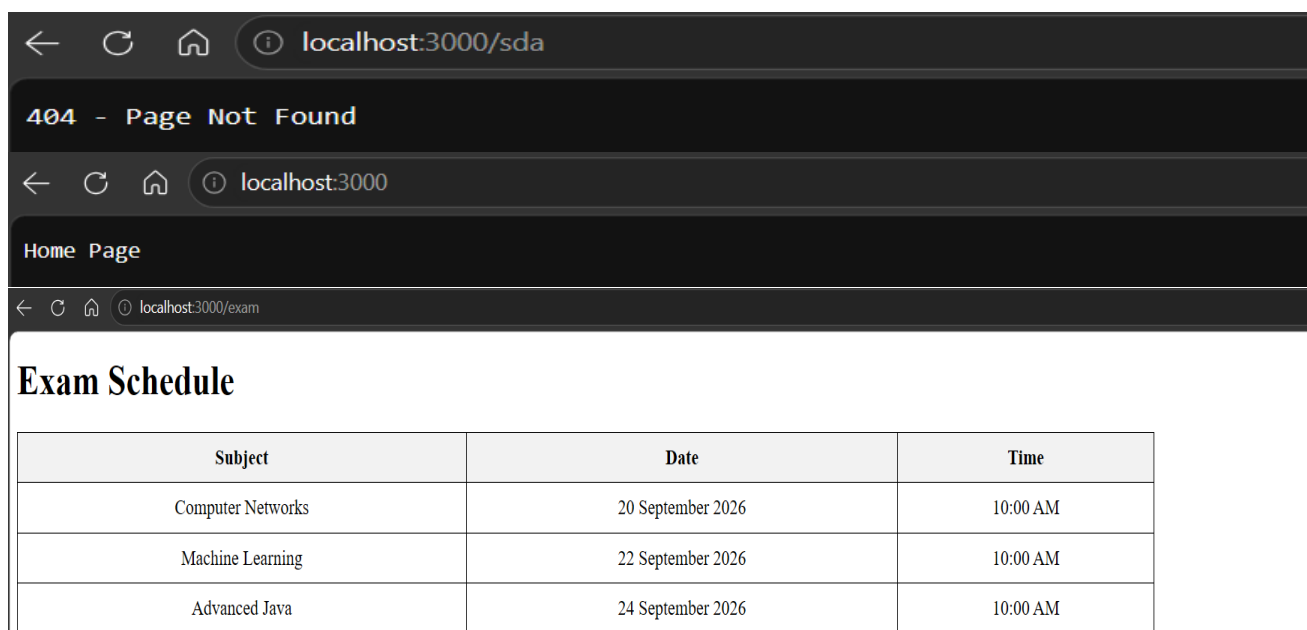
        <table>
          <tr>
            <th>Subject</th>
            <th>Date</th>
```

```

        <th>Time</th>
      </tr>
      <tr>
        <td>Computer Networks</td>
        <td>20 September 2026</td>
        <td>10:00 AM</td>
      </tr>
      <tr>
        <td>Machine Learning</td>
        <td>22 September 2026</td>
        <td>10:00 AM</td>
      </tr>
      <tr>
        <td>Advanced Java</td>
        <td>24 September 2026</td>
        <td>10:00 AM</td>
      </tr>
    </table>
  </body>
</html>
`);
} else {
  res.writeHead(404, { 'Content-Type': 'text/plain' });
  res.end('404 - Page Not Found');
}
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});

```

Output:


The screenshot shows a web browser interface. The top part displays a 404 - Page Not Found error for the URL localhost:3000/sda. Below this, the browser shows the Home Page for localhost:3000. The main content area displays an 'Exam Schedule' page with a table containing three rows of exam information.

Subject	Date	Time
Computer Networks	20 September 2026	10:00 AM
Machine Learning	22 September 2026	10:00 AM
Advanced Java	24 September 2026	10:00 AM

2: Build a Node.js HTTP server that takes a string via URL parameters and returns the reversed string:

- Example: `/reverse?text=hello` returns “olleh”

Code:

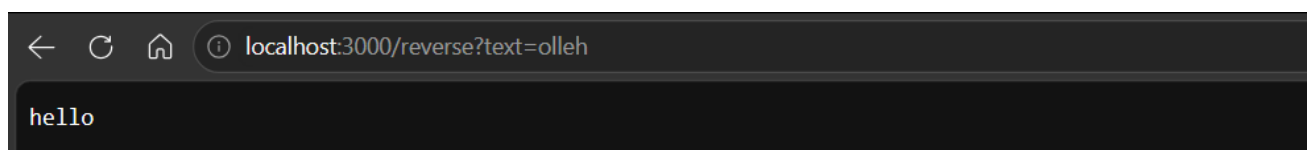
```
const http = require('http');
const { URL } = require('url');
const server = http.createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  if (url.pathname === '/reverse') {
    const text = url.searchParams.get('text');
    if (text === null) {
      res.writeHead(400, { 'Content-Type': 'text/plain' });
      res.end('Please provide a text parameter.');
```

```
      return;
    }
    const reversed = text.split('').reverse().join('');
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(reversed);
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 - Page Not Found');
```

```
  }
});
server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
```

```
});
```

Output:



3: Create a module “TextProcessing” contain functions named countWords, countLines, countCharacters that takes a string as input from ‘file1.txt’ file and returns the total number of words, lines and characters in the console.

Code TextProcessing.js:

```
function countWords(text) {  
  return text.trim().split(/\s+/).length;  
}
```

```
function countLines(text) {  
  return text.split(/\r?\n/).length;  
}
```

```
function countCharacters(text) {  
  return text.length;  
}
```

```
module.exports = {  
  countWords,  
  countLines,  
  countCharacters,  
};
```

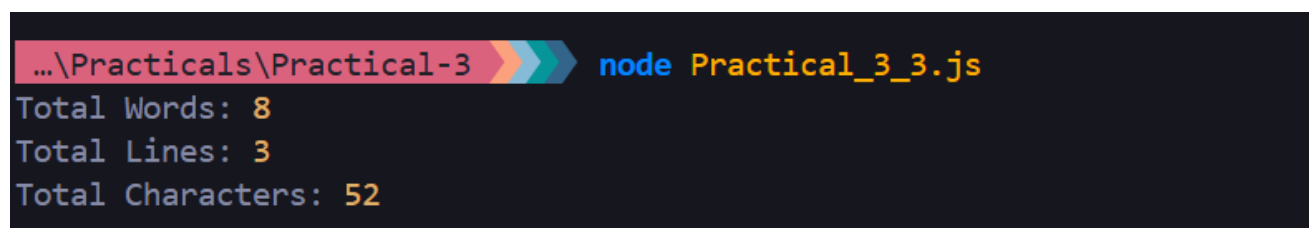
Code file1.txt:

```
Hello World  
This is Node.js  
Text Processing Module
```

Code:

```
const fs = require('fs');  
const TextProcessing = require('./TextProcessing');  
const text = fs.readFileSync('file1.txt', 'utf8');  
console.log("Total Words:", TextProcessing.countWords(text));  
console.log("Total Lines:", TextProcessing.countLines(text));  
console.log("Total Characters:", TextProcessing.countCharacters(text));
```

Output:



```
...\\Practicals\\Practical-3 >>> node Practical_3_3.js  
Total Words: 8  
Total Lines: 3  
Total Characters: 52
```

4: Demonstrate synchronous and asynchronous file operations with an example.

Code:

```
const fs = require('fs');

// Synchronous Operation
console.log('Synchronous File Read:');
const syncData = fs.readFileSync('file1.txt', 'utf8');
console.log(syncData);
console.log('Synchronous operation completed.');
```

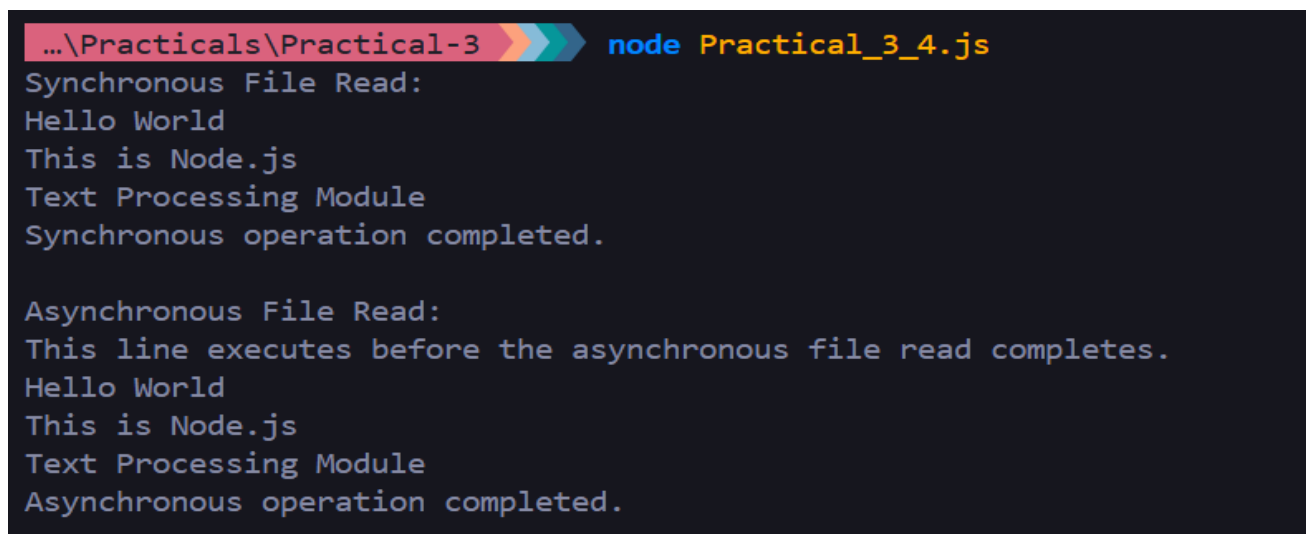


```
// Asynchronous Operation
console.log('\nAsynchronous File Read:');
fs.readFile('file1.txt', 'utf8', (err, data) => {
  if (err) {
    console.log('Error:', err);
    return;
  }
  console.log(data);
  console.log('Asynchronous operation completed.');
```



```
});
console.log('This line executes before the asynchronous file read completes.');
```

Output:



```
... \Practicals \Practical-3 >>> node Practical_3_4.js
Synchronous File Read:
Hello World
This is Node.js
Text Processing Module
Synchronous operation completed.

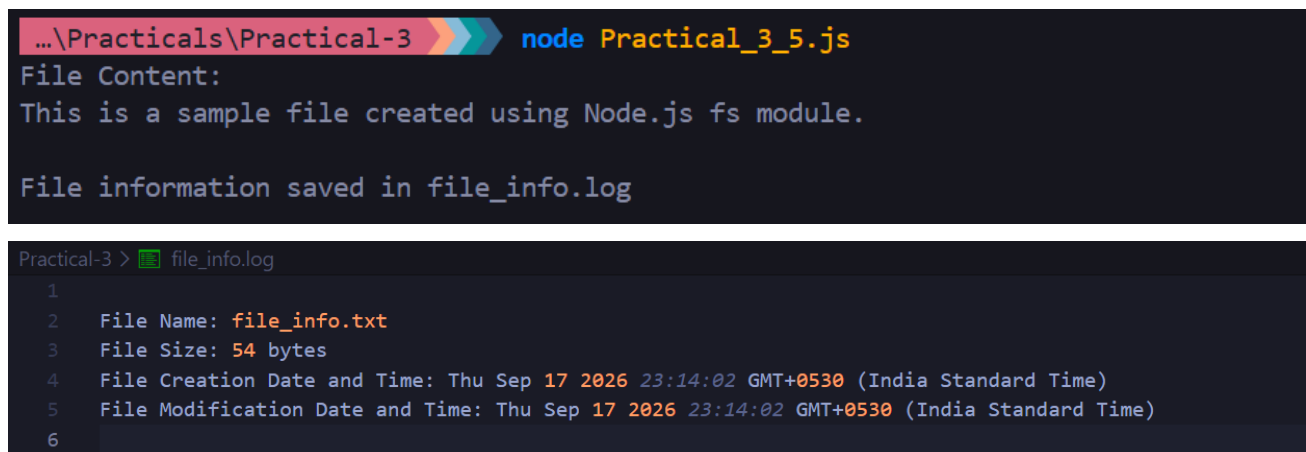
Asynchronous File Read:
This line executes before the asynchronous file read completes.
Hello World
This is Node.js
Text Processing Module
Asynchronous operation completed.
```

5: Create a file named file_info.txt and write some content in it. Implement a function that reads the content of file_info.txt using the fs module and create log file to maintain information about File name, File size (bytes), File creation date and time, File modification date and time.

Code:

```
const fs = require('fs');
fs.writeFileSync(
  'file_info.txt',
  'This is a sample file created using Node.js fs module.',
);
```

```
function getFileInfo() {
  const content = fs.readFileSync('file_info.txt', 'utf8');
  console.log('File Content:');
  console.log(content);
  const stats = fs.statSync('file_info.txt');
  const log = `
File Name: file_info.txt
File Size: ${stats.size} bytes
File Creation Date and Time: ${stats.birthtime}
File Modification Date and Time: ${stats.mtime}
`;
  fs.writeFileSync('file_info.log', log);
  console.log(`\nFile information saved in file_info.log`);
}
getFileInfo();
```

Output:


```
...\\Practicals\\Practical-3 > node Practical_3_5.js
File Content:
This is a sample file created using Node.js fs module.

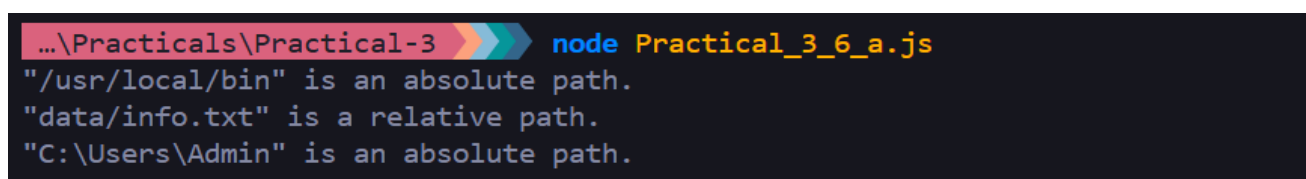
File information saved in file_info.log

Practical-3 > file_info.log
1
2   File Name: file_info.txt
3   File Size: 54 bytes
4   File Creation Date and Time: Thu Sep 17 2026 23:14:02 GMT+0530 (India Standard Time)
5   File Modification Date and Time: Thu Sep 17 2026 23:14:02 GMT+0530 (India Standard Time)
6
```

6: Check the output of following code:

Code A:

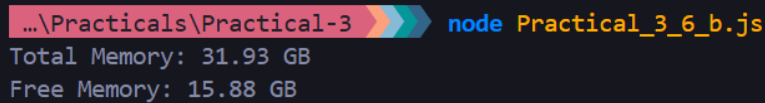
```
const path = require('path');
function checkPathType(filePath) {
  if (path.isAbsolute(filePath)) {
    console.log(`${filePath} is an absolute path.`);
  } else {
    console.log(`${filePath} is a relative path.`);
  }
}
// Example
checkPathType('/usr/local/bin');
checkPathType('data/info.txt');
checkPathType('C:\\Users\\Admin');
```

Output A:


```
...\\Practicals\\Practical-3 > node Practical_3_6_a.js
"/usr/local/bin" is an absolute path.
"data/info.txt" is a relative path.
"C:\\Users\\Admin" is an absolute path.
```

Code B:

```
const os = require('os');
function displayMemoryInfo() {
  const freeMem = os.freemem();
  const totalMem = os.totalmem();
  console.log('Total Memory:', (totalMem / 1024 ** 3).toFixed(2), 'GB');
  console.log('Free Memory:', (freeMem / 1024 ** 3).toFixed(2), 'GB');
}
// Example usage
displayMemoryInfo();
```

Output B:


```
...\\Practicals\\Practical-3 > node Practical_3_6_b.js
Total Memory: 31.93 GB
Free Memory: 15.88 GB
```

7: Do as directed:

- a) Register a listener with customize event.
- b) Create multiple listeners with specific event.
- c) Remove a specific listener.
- d) Remove all listeners.
- e) How to get maximum size of listener will bind with specific event?
- f) How to change maximum size of listeners to bind with specific event?
- g) Find out name of active listeners for the specified event.
- h) Find out number of listeners listening to the specified event.
- i) How to change execution position of listeners?

Code:

```
const EventEmitter = require('events');
const eventEmitter = new EventEmitter();

// a. Register a listener with custom event
function welcomeListener() {
  console.log('Welcome to Node.js!');
}
eventEmitter.on('welcome', welcomeListener);
console.log('a. Custom event:');
eventEmitter.emit('welcome');

// b. Create multiple listeners for a specific event
function listener1() {
  console.log('Listener 1 executed');
}
```

```
function listener2() {
  console.log('Listener 2 executed');
}
function listener3() {
  console.log('Listener 3 executed');
}

eventEmitter.on('message', listener1);
eventEmitter.on('message', listener2);
eventEmitter.on('message', listener3);

console.log('\nb. Multiple listeners:');
eventEmitter.emit('message');

// c. Remove a specific listener
eventEmitter.removeListener('message', listener2);
console.log('\nc. After removing Listener 2:');
eventEmitter.emit('message');

// d. Remove all listeners
eventEmitter.removeAllListeners('message');

console.log('\nd. After removing all message listeners:');
console.log('Number of listeners:', eventEmitter.listenerCount('message'));

// e. Get maximum number of listeners
console.log('\ne. Maximum listeners allowed:');
console.log(eventEmitter.getMaxListeners());

// f. Change maximum number of listeners
eventEmitter.setMaxListeners(20);

console.log('\nf. New maximum listeners:');
console.log(eventEmitter.getMaxListeners());

// g. Find names of active listeners
eventEmitter.on('login', function userLogin() {
  console.log('User logged in');
});
eventEmitter.on('login', function adminLogin() {
  console.log('Admin logged in');
});
console.log('\ng. Active listener names:');
const listeners = eventEmitter.listeners('login');
listeners.forEach((listener) => {
  console.log(listener.name);
});

// h. Find number of listeners for an event
console.log('\nh. Number of login listeners:');
```

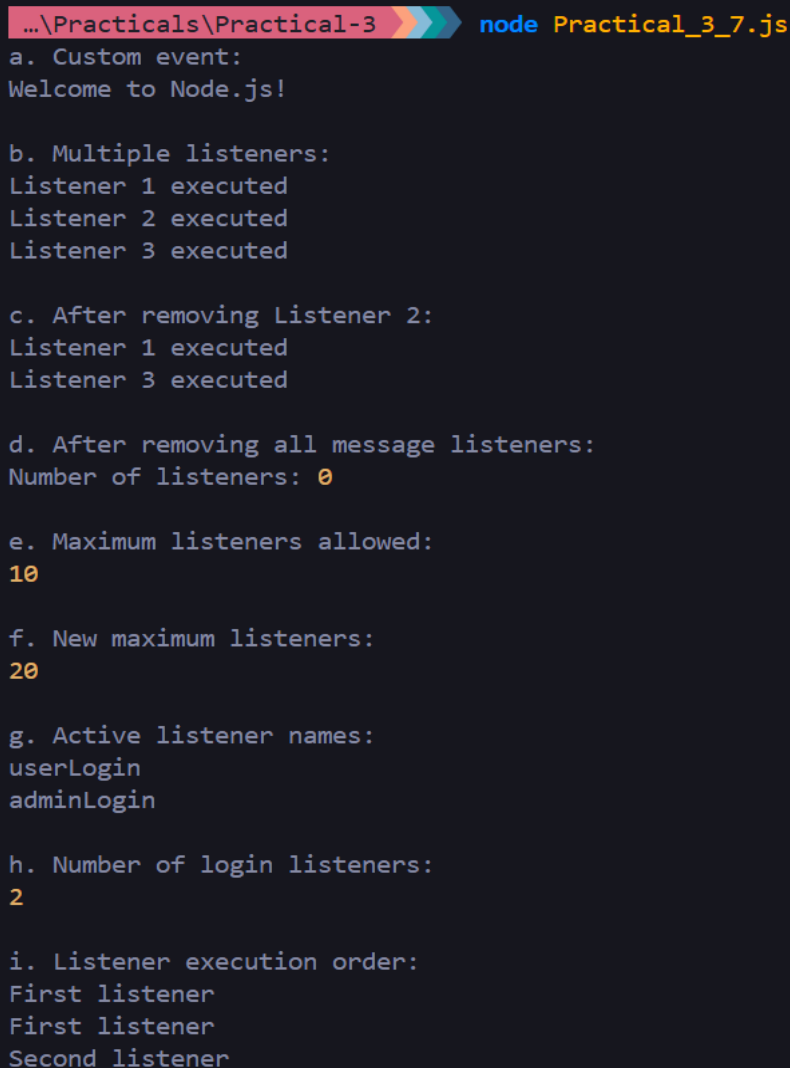
```
console.log(eventEmitter.listenerCount('login'));

// i. Change execution position of listeners

function firstListener() {
  console.log('First listener');
}
function secondListener() {
  console.log('Second listener');
}
eventEmitter.on('order', firstListener);
eventEmitter.on('order', secondListener);

// Move firstListener to the beginning
eventEmitter.prependListener('order', firstListener);

console.log("\ni. Listener execution order:");
eventEmitter.emit('order');
```

Output:

```
...\\Practicals\\Practical-3 >>> node Practical_3_7.js
a. Custom event:
Welcome to Node.js!

b. Multiple listeners:
Listener 1 executed
Listener 2 executed
Listener 3 executed

c. After removing Listener 2:
Listener 1 executed
Listener 3 executed

d. After removing all message listeners:
Number of listeners: 0

e. Maximum listeners allowed:
10

f. New maximum listeners:
20

g. Active listener names:
userLogin
adminLogin

h. Number of login listeners:
2

i. Listener execution order:
First listener
First listener
Second listener
```